01 / 先看结论与边界

从一条盘点帖到四份可运行工具

没有板子，先选对模拟器

官方模拟器 ≠ 官方 QEMU

12个问题 · 6步主实验 · 真实串口可重放

已实读原帖全文、作者回复和三个仓库，并在云端真跑了官方预编译固件。

主角是 Velxio、esp32sim、乐鑫 **esp-emulator**；评论区另补出 **espressif/qemu**。作者明确说「官方是第三个」，因此不能把 QEMU 叉和 esp-emulator 当成同一个产品。[S1]

> **状态：学习交付完成 / 网页交互、真板对照、作者私有固件待验证**
> 本批真实运行了 esp-emu v1.0.0-rc2、乐鑫 QEMU 9.2.2（20261109）和 esp32sim 0.3.0。使用的是官方 examples 按键固件和仓库内公开 hello_world，不是作者自己的验收固件。未部署 Velxio（本机无 Docker），未打开网页编辑器做交互动画，没有实物板。

可以拿它做什么？（用途不是已部署成果）

1. **无板预验收：**把 merge 后的固件先在模拟器里看启动日志和串口，再决定买哪块芯片。
2. **CI 冒烟：**用 `--exit-on` / `--timeout` / `--max-seconds` 把 “Hello world!” 或按键边沿做成可退出的检查。
3. **按键与虚拟零件：**官方 examples 的 GPIO 按键可在 esp-emu 里用控制通道按/放，验证中断路径。
4. **镜像格式教学：**区分应用分区 .bin、merge-bin、以及 QEMU 要求的 2/4/8/16MB 填充。
5. **选型清单：**先问芯片（尤其经典 ESP32）、外设、是否要联网、是否要开源核心、网页还是 CLI。

2026-11-11 · Asia/Shanghai（UTC+8） | 本批只保存候选，不提交Git、不勾选任务。沿用旧指南的圆角问答卡、渐变标题、强调字与SVG，配色为鲜艳青/蓝/橙。HTML 不内嵌字体和大图。

02 / 从疑问走进盘点

Q01 原帖到底在荐什么？那张配图是对照表吗？

正文选出三款：**Velxio**（浏览器画电路，ESP 系列在服务器上跑 QEMU）、**esp32sim**（从芯片 ROM 启动，S3 最成熟）、**esp-emulator**（官方，芯片与协议最全）。作者称用自己的固件在官方模拟器上跑通点灯、双核、NVS、PSRAM、LVGL，但外设还要等。帖末预告自研「试板 TryBoard」。附件图片实看是 TryBoard 界面：ESP32-S3-DevKitC-1、GC9A01 圆屏、串口 TRYBOARD_OK / BLINK，不是三款开源工具的对照表。[S1]



> 帖中星标是 2026-10-19 的快照：3.3k / 364 / 190。本批读取日（2026-11-11）GitHub 为 3679 / 1126 / 536，另加 qemu 367。不要把旧星标写成当前规模。

Q02 乐鑫不是早就有 QEMU 吗？和官方模拟器是一回事吗？

不是。评论区有人问「官方不是有个 qemu」，作者回答「官方是第三个」，对方随即贴出 espressif/qemu。esp-emulator 是 2026 年新的 Rust 二进制模拟器，支持 C3/C5/C6/H2/P4/S3/S31，带网页版和 GDB；espressif/qemu 是带乐鑫补丁的 QEMU 叉，本批预编译 9.2.2 在 xtensa 上提供 esp32/esp32s3，在 riscv32 上提供 c3/c5/c6/c61/h2/h4/p4。二者都是乐鑫维护，但仓库、许可形态和机器模型都不同。[S1][S3][S4]



> 选型时把「乐鑫官方」拆成两行：要经典 ESP32 或开源机器模型，优先看 qemu；要官方文档里的芯片/协议清单和网页仪表盘，看 esp-emulator。

深入一层 / 芯片与镜像

Q03 经典 ESP32 为什么进不了官方模拟器?

esp-emu 的 `--chip` 合法值只有 esp32c3/c5/c6/h2/p4/s3/s31。我们实跑 `--chip esp32`，退出码 2，提示 invalid value，并建议相近的 esp32s3。这不是猜，是当前 v1.0.0-rc2 的 CLI 行为。对比：同一份公开经典 hello_world，在 QEMU `-machine esp32` 和 esp32sim `--chip esp32` 上都能打印「This is esp32 chip with 2 CPU core(s)」。esp32sim 作者也在回复里强调他们支持经典 ESP32 且核心开源。[S3][E2][E6]



> 手里是 ESP32-WROOM / DevKit V1 这类经典芯片，就不要把 esp-emulator 当第一选择。S3 和 C3 才是它的主场。

Q04 为什么不能把应用分区 .bin 直接丢给模拟器?

ESP-IDF 默认至少有三块：bootloader、分区表、app。esp-emu 要的是 **merge-bin 后的整片 flash**；评论也提醒过网页版同样如此。QEMU 还更严：镜像尺寸只能是 2/4/8/16MB。我们把 222224 字节的官方按键固件直接喂给 QEMU，报 `Drive size error`；用 0xFF 填到 4MB 后即可启动。C3 常见布局是 0x0 / 0x8000 / 0x10000，经典 ESP32 的 bootloader 在 0x1000。应用分区的 0xE9 魔数正确，也不等于整片镜像可用。[S3][S4][E4]



> 本批 esp-emu 可以直接吃 222224 字节的未填充合并镜像；QEMU 不行。同一份固件换工具，先看镜像契约，再谈「能不能跑」。

深入一层 / 网络、开源与保真

Q05 网页版是不是和本地版一样能上网?

原帖已经写清：官方网页版不联网，本地版可以连真实互联网。文档补充：浏览器构建把固件留在本机标签页，若要上网需另装 `esp-emu-relay`。esp32sim 网页版同样没有套接字，只能连虚拟 AP；本地可用 `--net nat` 访问真实网络。Velxio 的 ESP 系列本就在服务器端 QEMU 上跑。本批在 Linux 上启动 esp-emu 时，TAP 设备 `tap0` 没有权限，工具自动回退到 user-mode NAT（网关 192.168.4.1）。这次没有做 HTTP/Wi-Fi 接通测试，不能把「后端已就绪」写成「固件已经访问外网」。[S2][S3][E3]

网页：默认不上外网 → 本地：user / TAP → TAP 失败会回退

> 要验 Wi-Fi 重连或 HTTPS，选本地后端并单独设计实验。点灯、NVS、LVGL 这类用例碰不到网页版的联网限制。

Q06 官方模拟器开源了吗？LICENSE 文件算不算已经开源?

仓库有 Apache-2.0 的 LICENSE，但发行物主要是预编译二进制、安装脚本、文档和 examples。issue #10 问源码会不会公布；乐鑫维护者 mahavirj 在 2026-09-07 20:28:17（Asia/Shanghai）回复：**尚未决定是否公布源码**，可先提功能或缺陷 issue。到 2026-11-01 仍有人追问时间表，issue 保持 open。因此「有 LICENSE」不等于「核心源码可构建、可贡献外设模型」。相对地，esp32sim 是 MIT 源码，Velxio 是 AGPL（另有商业许可），qemu 整体为 GPL-2.0。[S3]

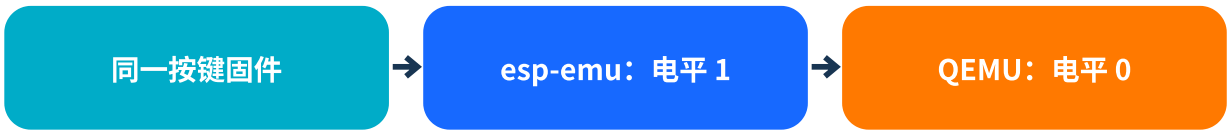
仓库有 LICENSE → 发行以二进制为主 → 官方：尚未决定

> 若你的目标是给缺失外设写模型并向上游合并，esp-emulator 目前不能当开源基础；qemu / esp32sim 更合适。

深入一层 / 实测对照

Q07 同一份按键固件，为什么默认电平不一样?

官方 examples 的 `gpio_button-esp32c3.bin` 在 esp-emu 里配合虚拟上拉按键零件，串口是 **BUTTON: ready level=1**；通过控制通道发送 press/release 后，依次出现 pressed 与 released，退出码 0。把同一镜像填充到 4MB 后交给 QEMU，应用同样启动，但变成 **BUTTON: ready level=0**——QEMU 没有这份 board.json 里的上拉零件。能启动 ≠ 外设默认状态一致。这正好呼应原帖「外设这块还得等等」，也说明虚拟零件是官方模拟器相对 QEMU 的实际加分项。[E3][E4]



> 不要把「串口出现 BUTTON: ready」直接等同于「和真板一样」。先核对上拉、复位strapping 和未建模的 I2C 器件。

Q08 芯片版本号对不上，是不是固件坏了?

同一份公开 C3 hello_world（ELF SHA 前缀 4812b1c53，IDF v5.5.4，堆 331296 字节）在三个模拟器里都打印了 **Hello world!**。但硅版本不同：esp-emu 与 QEMU 报 v0.3，esp32sim 报 v0.4；闪存芯片分别是 gd / gd / generic。经典 ESP32 同一份镜像（SHA 前缀 262a8b0a1，堆 303972）在 QEMU 报 v0.0，在 esp32sim 报 v2.0 并出现 XTAL 警告（Bogus XTAL freq: 8MHz），应用仍跑通。这些是默认 eFuse/时钟模型差异，不是「固件坏了」。esp32sim 自己把经典芯片标成 early，警告应当保留在记录里。[E5][E6]



> 若业务代码依赖特定芯片修订或 eFuse，要先用工具提供的 eFuse 镜像对齐，再比较行为；不要默认「模拟器里的 rev 就是你手上那颗硅」。

深入一层 / 怎么选、还缺什么

Q09 许可和部署方式会怎样限制使用?

Velxio 核心是 AGPL-3.0，另提供商业许可；自建 Docker 镜像覆盖 Arduino / Pico / ESP32 家族，但文档写明 STM32、树莓派 Linux、ESP32-C6 和部分合作厂商板只在 velxio.dev（有的还要付费桌面版）。esp32sim 核心 MIT，演示资源各有许可，mask ROM 来自乐鑫 Apache-2.0 的 esp-rom-elfs，不在发布包内。esp-emulator 二进制按仓库 LICENSE 使用，源码政策未定。qemu 整体 GPL-2.0。本批未部署 Velxio，也没有把这些许可翻译成法律意见。[S2][S3][S4]



> 「能在浏览器打开」和「能离线自建全部 40 块板」不是同一句话。先读自建镜像的板卡清单。

Q10 作者说的点灯、双核、NVS、PSRAM、LVGL，我们复现了吗?

没有。那是作者用**自己的验收固件**对官方模拟器的报告，原帖未给出该固件或仓库。本批跑的是：官方 examples 按键固件（C3，单核日志写 Unicore app）、公开 hello_world（C3 单核、经典 ESP32 双核）。经典双核 hello 出现在 QEMU 与 esp32sim，而不是 esp-emu。没有跑 NVS 读写、PSRAM 分配或 LVGL 刷新，不能把作者清单升级为本批实测。作者关于时序/采集精度模拟不了、网络可模拟重连的回复，也未点名具体工具，只能当访谈线索。[S1]



> 双核在经典 ESP32 与 S3 上才是默认故事；C3 本来就是单核。看「双核通过」时先确认芯片。

深入一层 / 怎么选、还缺什么

Q11 没有板子时，怎样按需求初选？

先写四列：芯片、外设、联网、部署。经典 ESP32 → qemu 或 esp32sim（或 Velxio 的 ESP32 板），不要选 esp-emu。要面包板/示波器/拖线 → Velxio（或等待 TryBoard，后者本批未运行）。要屏幕/触摸/摄像头/音频且偏 S3 → esp32sim。要 C6/H2 Thread、GDB、官方协议文档 → esp-emu 或 qemu（按外设清单逐项核对）。要进 CI、只要串口断言 → esp-emu 与 esp32sim 的超时/退出开关都可用。最后才看星标和网页截图。[S2][S3]

先写芯片和外设 → **再看联网与部署** → **最后才看星标**

> 总览里没有 ESP 模拟器专条；相邻的是 zigbee2mqtt 智能家居，以及并行批次可能出现的 ESP32/Tailscale 网络笔记。本批补的是模拟器选型，不重写家庭组网。

Q12 哪些问题仍然必须上真板，或至少再做网页实验？

供电、上拉电阻误差、天线与认证、接触不良、时序边沿、未建模的 I2C/SPI 器件，都不是本批串口实验能签字的。本批也没有：Velxio 画布交互、三个官方网页 playground、作者私有固件、PSRAM/LVGL/NVS 专项、真实 Wi-Fi 关联。QEMU 预编译依赖 libslirp；本机 apt 源异常，我们从 Ubuntu 包提取了 libslirp.so.0 才启动，这是环境事实，不是 QEMU 缺陷本身。云端不能访问用户本机磁盘，不构成停工理由。[E1][E4]

模拟器：启动与逻辑
本批已跑
→ **真板：电源与射频**
待验证
→ **网页画布交互**
待验证

> 把模拟器当预验收和 CI，把真板当签约测试。两者都需要同一份固定固件哈希，才能比较。

03 / 主实验：从环境到按键

教学脚本在解压 exercise ZIP 后运行：`python3 exercise/lab.py env|matrix|flash|verify`。它不启动模拟器。下面 E2–E6 的命令在本批云端真实执行，工具版本已固定；完整标准输出见 experiment-log.md。

E1 环境、校验和能力边界

目的：记下 Python/CPU、校验预编译包，并承认没有 Docker、没有 NVIDIA、没有实物板。

```
python3 exercise/lab.py env
sha256sum -c SHA256SUMS          # 各发布页提供
qemu-system-riscv32 --version    # 本批: 9.2.2 (esp_develop_9.2.2_20261109)
esp-emu --version                # 本批: 1.0.0-rc2
./esp32sim --version             # 本批: 0.3.0
```

真实结果：Python 3.13.5，x86_64，无 Docker。三个预编译包与官方 SHA256 一致。QEMU 因缺少 libslirp.so.0 无法启动；从 Ubuntu `libslirp0_4.7.0-1ubuntu3_amd64.deb` 抽出库后，用 LD_LIBRARY_PATH 启动成功。本机 apt 源返回 500，没有用它安装。

验证：核对版本字符串与校验和；库workaround 必须写进日志，不能假装「解压即可运行」。

E2 官方模拟器拒绝经典 ESP32，并启动按键固件

目的：用 CLI 证实芯片缺口，再用官方 examples 固件看到真实 ROM 启动串口。

```
esp-emu --chip esp32 --firmware gpio_button-esp32c3.bin --timeout 1s
# exit=2, invalid value 'esp32'
esp-emu --chip esp32c3 --firmware gpio_button-esp32c3.bin \
    --cosim-board board.json --timeout 8s --exit-on "BUTTON: ready"
```

真实结果：拒绝经典芯片。第二次启动打印 ESP-ROM:esp32c3-api1-20210207、IDF v6.1 引导、Project name: gpio_button，然后 `BUTTON: ready level=1`，退出码 0。TAP 无权限，自动回退 user-net。

验证：退出码与串口关键字；这是官方 examples，不是作者私有固件。

03 / 主实验：对照比“能启动”重要

E3 虚拟按键的按下与释放

目的：证明控制通道能驱动固件自己的 GPIO 中断路径。

```
esp-emu --chip esp32c3 --firmware gpio_button-esp32c3.bin \
    --cosim-board board.json --control-tcp 127.0.0.1:5556 \
    --timeout 20s --exit-on "BUTTON: released"
# 另一进程在 ready 之后发送:
# part list / part button input press / part button input release
```

真实结果：控制通道依次回答 `ok button`、ok、ok。串口顺序为 ready → pressed → released，退出码 0。

验证：三种字符串都出现且顺序正确。未与真板按键抖动或上拉电阻对比。

E4 同一镜像在 QEMU 上：先失败，再填充，再看默认电平

目的：验证尺寸契约，并对照没有虚拟零件时的 GPIO 默认值。

```
qemu-system-riscv32 -nographic -icount 3 -machine esp32c3 \
    -drive file=gpio_button-esp32c3.bin,if=mtd,format=raw
# Drive size error: only 2, 4, 8, and 16MB images are supported
# 将镜像 0xFF 填充到 4MB 后再跑同一条命令
```

真实结果：未填充退出码 1。填充后启动到 `BUTTON: ready level=0`（不是 1），并警告检测到 4096k 大于镜像头 2048k。因 QEMU 不会按字符串自动退出，本批用 timeout 结束（124）。

验证：同一哈希固件、两种工具、两种默认电平。这是外设模型差异，不是「其中一个没跑起来」。

03 / 主实验：公开 hello 与经典芯片

E5 同一份公开 C3 hello_world，三工具对照

目的：用仓库内公开分段镜像（bootloader+ptable+app，IDF v5.5.4）检查「Hello world!」。C3 拼到 0x0/0x8000/0x10000 并填 4MB；esp32sim 直接吃三段。

```
qemu-system-riscv32 -nographic -icount 3 -machine esp32c3 \
    -drive file=c3-hello-merged-4mb.bin,if=mtd,format=raw
esp-emu --chip esp32c3 --firmware c3-hello-merged-4mb.bin \
    --timeout 6s --exit-on "Hello world!"
esp32sim-c3 --boot rom --rom esp32c3_rev3_rom.elf \
    --bootloader c3-hello-bootloader.bin --ptable c3-hello-ptable.bin \
    --app c3-hello_world.bin --max-seconds 8 --no-dump
```

真实结果：三份都打印 Hello world!，堆均为 331296。修订号：esp-emu/QEMU = v0.3，esp32sim = v0.4。闪存标识 gd / gd / generic。

验证：同一 ELF 前缀 4812b1c53。这是公开示例，不是本批现编译的 IDF 工程（本机无 Docker/IDF 工具链）。

E6 经典 ESP32 hello_world：QEMU 与 esp32sim

目的：在 esp-emu 无法接受 `--chip esp32` 的前提下，验证另外两个工具能跑经典双核示例。镜像布局为 bootloader@0x1000。

```
qemu-system-xtensa -nographic -icount 3 -machine esp32 \
    -drive file=classic-hello-merged-4mb.bin,if=mtd,format=raw
esp32sim --chip esp32 --board esp32dev --boot rom \
    --rom esp32_rev300_rom.elf \
    --bootloader classic-hello-bootloader.bin \
    --ptable classic-hello-ptable.bin \
    --app classic-hello_world.bin --max-seconds 8 --no-dump
```

真实结果：两者都打印 Hello world! 和「esp32 chip with 2 CPU core(s)」，堆 303972。QEMU 报硅版本 v0.0；esp32sim 报 v2.0，并出现 XTAL 警告（Bogus XTAL freq: 8MHz），应用仍继续。esp32sim 报告两个核都执行了指令。

验证：同一 ELF 前缀 262a8b0a1。警告必须保留，不能把 early 支持写成与硅片完全一致。

04 / 合入候选与待验证清单

> 主实验使用的是公开预编译固件和官方 examples，不是上游测试套件、不是作者私有验收包，也没有 ESP-IDF Docker 现编译。数字不能用来推断 LVGL 帧率、Wi-Fi 吞吐或真板时序。verify 的 8 组检查是同一 worker 自写脚本，不能称独立审核。

增量，而非重写旧知识库

总览固定在 a8674777e18302b8804b137212b1396cbfe5a747（2026-11-04 17:42:58，Asia/Shanghai）。已检索树和总览：没有 Velxio / esp32sim / esp-emulator / 乐鑫 QEMU 专条；命中的相近领域是物联网/zigbee2mqtt，主题不重叠。本批只建议新增模拟器选型笔记，不改历史累计数量。

版本固定

来源	固定版本与范围
原帖	x.com/wquguru/status/2112084878842085436，2026-10-19 15:34:10（Asia/Shanghai）；note_tweet 全文+配图实看+作者回复。
Velxio	master 8a254109afe4cd17e91fb0aeaabf14ab42c46c4a，2026-11-10 11:29:12（Asia/Shanghai）。读 README/LICENSE/自建说明。未运行。
esp32sim	源码 main 3d049f5c4df9efefb76406601db7a38661c328b6；运行的是发布包 v0.3.0。ROM：esp-rom-elfs 20260528。
esp-emulator	源码/文档 f77b06d76ec1c8323c21d3543ec86bf426567f22；运行 v1.0.0-rc2 官方二进制与 examples。
espressif/qemu	文档 cdee381dce7b88b5207ec48c72984ca31d495dd3；运行预编译 9.2.2_20261109。

待验证（不属于已执行主实验）

Velxio 自建/网页画布；三个官方 playground 的交互操作；作者私有验收固件（点灯/NVS/PSRAM/LVGL）；真实 Wi-Fi/BLE/Thread；与实物板的逐项对照。若要补编译，需独立的 ESP-IDF 工具链或 Docker 环境。

- [S1] https://x.com/wquguru/status/2112084878842085436 及作者回复 2112222077671673864、2112118583698526573
- [S2] https://github.com/davidmonterocrespo24/velxio/tree/8a254109afe4cd17e91fb0aeaabf14ab42c46c4a
- [S3] https://github.com/mikroverk/esp32sim/tree/3d049f5c4df9efefb76406601db7a38661c328b6；https://github.com/espressif/esp-emulator/tree/f77b06d76ec1c8323c21d3543ec86bf426567f22；issue #10
- [S4] https://github.com/espressif/qemu；https://github.com/espressif/esp-toolchain-docs/blob/cdee381dce7b88b5207ec48c72984ca31d495dd3/qemu/esp32c3/README.md

详细 URL、SHA、日期与本地指纹见 sources.md；合入建议见 knowledge-proposal.md；完整真实输出见 experiment-log.md。本文由同一 worker 自查，不声称独立审核。未更改共享 vault、用户任务或日程。